

Algorithm for file updates in Python

Project description

In this project, I developed an algorithm in Python to automate the updating of an IP address-based access control list in a healthcare environment. The main objective was to compare an existing allow list (allow_list.txt) with a provided remove list and automatically remove any unauthorized access. This solution reduces the probability of manual errors and ensures that only verified personnel maintain access to the restricted subnet with patient records. Through this process, I integrated file management (I/O) with programming logic to strengthen the organization's security posture.

Open the file that contains the allow list

To begin, I assigned the file name to the variable `import_file`. Then, I used the `with` statement along with the `open()` function to access the file securely.

```
import_file = "allow_list.txt"

with open(import_file, "r") as file:
    # The code continues here
```

Explanation: The keyword `def` is not necessary here, but the `with` statement is fundamental because it ensures that the file is closed automatically after the code block finishes executing, preventing memory leaks. The `open()` function takes the variable `import_file` and the mode `"r"` (read). I used `file` as the context variable to handle the file object within the statement.

Read the file contents

Once the file was opened, I used the `.read()` method to extract the content and convert it into a string that Python can process.

```
ip_addresses = file.read()
```

Explanation: The `.read()` method takes all the content from the file object `file` and stores it as a single string in the variable `ip_addresses`. This is necessary to be able to visualize and subsequently transform the data contained in the text file.

Convert the string into a list

To facilitate the manipulation of individual IP addresses, I transformed the string into a list.

```
ip_addresses = ip_addresses.split()
```

Explanation: I used the `.split()` method on the variable `ip_addresses`. By default, this method divides a string into individual elements based on whitespace (including line breaks) and places them within a list. This allows iterating over each IP address as an independent element.

Iterate through the remove list

I set up a for loop to iterate through each of the IP addresses that should be removed, which are found in `remove_list`.

```
for element in remove_list:  
    # Removal logic here
```

Explanation: The loop header uses the keyword `for` followed by the loop variable `element`. This loop will repeat as many times as there are elements in `remove_list`, allowing each IP address candidate to be evaluated for removal from the access list.

Remove IP addresses that are on the remove list

Inside the loop, I implemented a conditional statement to verify if the IP address from the remove list is in our allow list.

```
if element in ip_addresses:  
    ip_addresses.remove(element)
```

Explanation: I used the keyword `if` along with the membership operator `in` to verify if `element` exists within `ip_addresses`. If the condition is true, the `.remove(element)` method is applied to the list. Applying the `.remove()` method in this way is possible and safe because there are no duplicates in the `ip_addresses` list.

Update the file with the revised list of IP addresses

To finish, I converted the updated list back into a string and overwrote the original file with the new data.

```
ip_addresses = "\n".join(ip_addresses)

with open(import_file, "w") as file:
    file.write(ip_addresses)
```

Explanation: First, I used the `.join()` method applied to the string `"\n"`, which concatenates all the elements of the `ip_addresses` list separating them by a new line. Then, I opened the file again using `with open()` but this time with the mode `"w"` (write). The `.write()` method takes care of replacing the previous content of the file with the updated string.

Summary

This algorithm effectively automates the management of network permissions by processing text files in Python. The script starts by opening and reading a list of authorized IP addresses, transforming that data from a string to a list to allow for manipulation. Through a for loop and an if conditional statement, the algorithm systematically identifies and removes IP addresses that should no longer have access. Finally, the cleaned data is converted back into a string and written to the original file, ensuring that access control is always up to date. This methodology provides a scalable and accurate solution for maintaining security in the company's infrastructure.